Sistema Bancario TCP Multithread

Hands-On 14 - MidTerm

Massimo Mantineo (541924)

Università degli Studi di Messina

A.A. 2024/2025

Indice

1. Introduzione al problema
2. Stato dell'arte
3. Metodologia usata
4. Risultati sperimentali
5. Conclusione e sviluppi futuri

Introduzione al problema

Obiettivo del MidTerm

Sviluppare un sistema client-server bancario per un singolo correntista.

- **Protocollo**: TCP (garanzia di consegna e ordine).
- **Accessi Concorrenti**: E-banking, App, Terminale, ecc.
- **Limiti Strutturali**: Massimo 10 connessioni threads globali simultanee elaborabili per volta per non sovraccaricare il server.
- **Operazioni Supportate**: CRUD su memoria volatile (ADD, DELETE, UPDATE, LIST).

Stato dell'arte

Le Componenti del Networking

- **TCP Sockets**: Modello di elaborazione che richiede `bind`, `listen` ed `accept`.
- **Paradigma Demultiplexer**: Il socket server master ha il solo scopo di instradare. Rilascia per ogni richiesta un nuovo file descriptor virtuale allocato specificatamente per il worker.
- **Race Conditions e Thread Safety**: I threads condividono il medesimo spazio di memoria dell'eseguibile master. Leggere e modificare simultaneamente la base di dati genererà inevitabilmente corruzioni in memoria.

Strutturazione Backend C

- Un array esteso `Transaction db[MAX_RECORDS]` opera da surrogato al database relazionale.
- Si utilizza un semaforo contatore (`sem_t limiter`) posizionato a limite 10. Viene chiamata la `wait()` prima della `accept()` per interdire nuovi allocamenti ad array già pieno.
- In fase di chiusura socket del client nel demone (`QUIT`), la chiamata a `post()` ripulisce il posto.

L'imperativo della Mutua Esclusione

L'assoluta stabilità dei dati è garantita dal *Mutex*:

- Quando stringiamo i comandi testuali tramite `sscanf`, filtriamo l'input.
- Prima di scorrere il database (`for loop`), **acquisiamo il lock**.
- In base al target (*Aggiunta, Rimozione o Modifica per ID*), salviamo il match, verifichiamo la clausola *Active = 1 o 0* della struct.
- **Rilasciamo il lock** solo a manipolazione della memoria conclusa, o durante un ritorno di Errore.

Risultati sperimentali

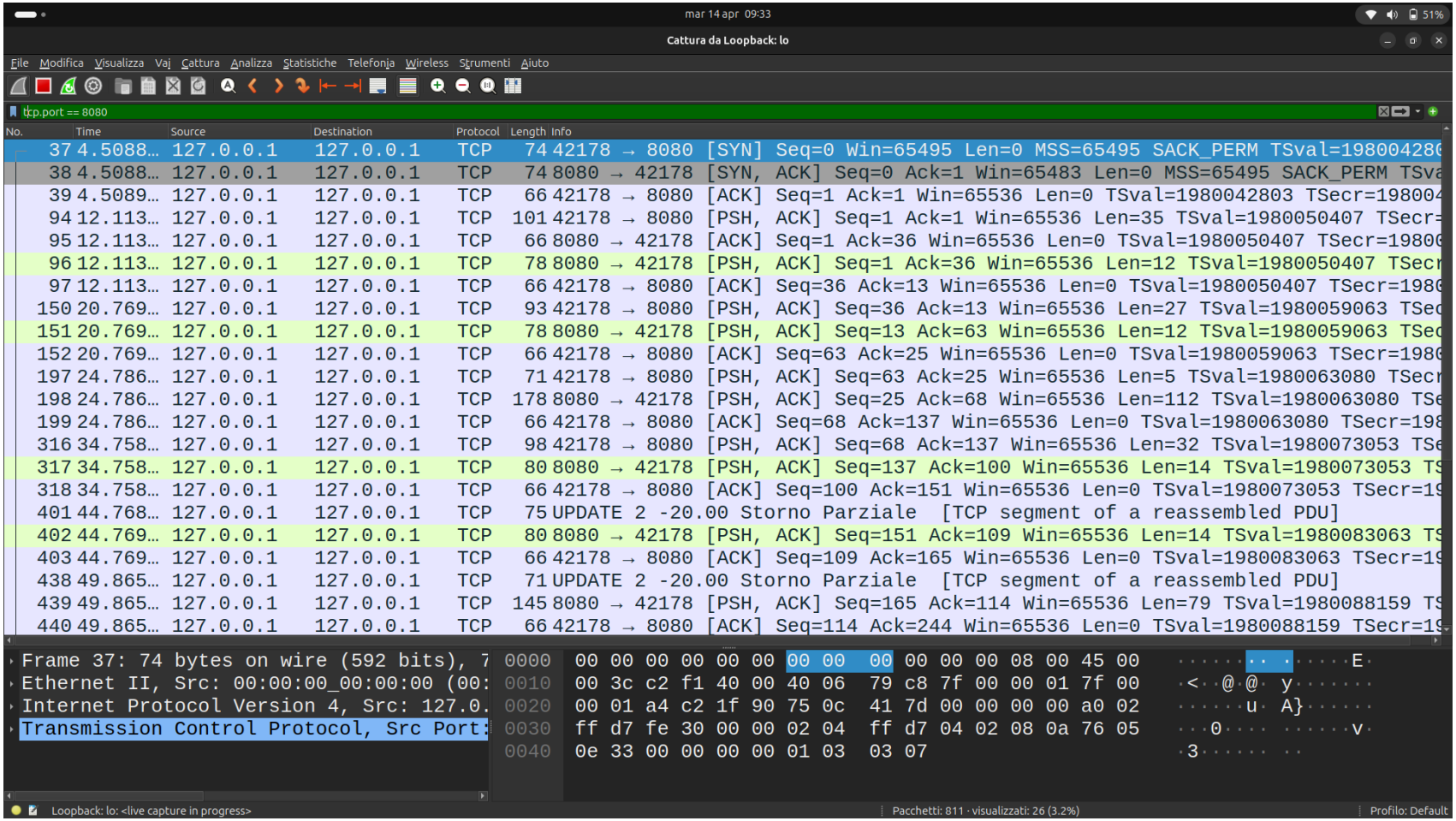
Testato con successo su terminale locale `127.0.0.1`.

```
--- Connected to Bank Server ---
BANK> ADD 101 25.50 Ricarica Prepagata
OK: ADDED 101
BANK> LIST
--- TRANSACTION LIST ---
[101] 25.50 - Ricarica Prepagata
-------------------------
```

Dimostrata anche la stabilità su accessi paralleli inviando in stallo il lock per emulazione. Nessun crash verificato.

Analisi del Traffico (Wireshark)

3-Way Handshake TCP



- Instaurazione canale tramite `[SYN]`, `[SYN, ACK]`, `[ACK]`.
- Trasferimento payload operativo segnalato via `[PSH, ACK]`.

Conclusione e sviluppi futuri

Conclusioni: Il codice sviluppato implementa un *Proof of Concept* robusto ed espandibile senza l'overhead di strutture applicative gravose.

Prospettive di Sviluppo Futuro:

- **Database Reale**: Sostituzione dell'array RAM con `SQLite/MariaDB` richiamati via Drivers C SQL per garantire persistenza (*Durability*).
- **Thread Pool**: Abbandono dei *Detached Threads* grezzi a favore di un pool pre-allocato di 10 worker, che estraggono i file-descriptor tramite coda (pattern Produttore-Consumatore appreso in HO12).

Grazie per l'attenzione!